The Staging Line Is Compactness: Behavioral Reification in Weak Combinatory Logic

N. E. DAVIS, ZeroAttest, USA

In weak combinatory logic over $\{S, K, I\}$ no term quotes every term, so a staged language built on it fixes quotation at compile time. We read that discipline as the S4 necessity modality of Davies–Pfenning and Kavvos, and ask where a *behavioral lift* exists: a closed term that sends a live value to a code agreeing with it, a weak analogue of Kavvos' Q-coalgebras and a weak substitute for the missing unit. The answer depends on where the value must be total. A lift exists at $\texttt{Bool} \to \texttt{Bool}$ under totality on the booleans; provably none exists at $\texttt{Nat} \to \texttt{Bool}$ under totality on the numerals. Under a modulus, an operational form of totality on a compact point set, a lift exists at $\texttt{Nat} \to \texttt{Bool}$ by search over $\mathbb{N}_\infty$ and at Cantor space, for functionals of streams, by Berger's search, which also gives one at $(\texttt{Nat} \to \texttt{Bool}) \to \texttt{Bool}$. Totality at the point at infinity suffices for $\texttt{Nat} \to \texttt{Bool}$, and totality on every point of Cantor space suffices in an oracle semantics; totality on the definable streams does not, by a Kleene tree built inside the calculus. The staging line for behavioral lift is therefore drawn at compactness of the point set on which totality is demanded, namely Escardó's boundary for decidable equality. We construct the modal reading and produce constructions in weak combinatory logic and a compactness observation. All theorems are mechanized in Lean 4.

CCS Concepts: • **Theory of computation** → **Lambda calculus**; *Modal and temporal logics*; *Equational logic and rewriting*; Constructive mathematics.

Additional Key Words and Phrases: combinatory logic, staging, modal type theory, reification, exhaustible sets, fan theorem, Kleene tree, Lean

1 Introduction

A language whose instruction set is $\{S, K, I\}$ cannot quote at run time: no closed term Q has $Q\,M \twoheadrightarrow \langle M \rangle$ for every M [Barendregt 1991; Davis 2027] (`no_quote`). The declarative *SKI* "assembly language" SKIjack [Davis 2026b] therefore makes quotation a compile-time construct, and checks that three surface positions (the operands of equality, namespace facts and scry paths) hold data, not functions; a quotation's body is checked against the program's symbol tables instead. Davies and Pfenning [Davies and Pfenning 2001] read staging as S4 necessity, and Kavvos [Kavvos 2017] develops intensional recursion in that frame for untyped λ, where evaluation and code-for-code are definable and only quoting is not. Section 3.1 places SKIjack's discipline in the same frame, with its counit and comultiplication mechanized over $\{S, K, I\}$.

While the syntactic unit is impossible, a weaker one need not be. A *behavioral* lift sends a live value to the code of *some* term that behaves as it does. We call the boundary between the live values lifted to codes by some closed term and those lifted by no closed term the *staging line*. Davies and Pfenning remark that a lift $A \to \Box A$ exists exactly when every arrow of A is under a box [Davies and Pfenning 2001, §5.3], a criterion suited to structural copies. MetaML forbids lifting functions by design [Taha and Sheard 2000], and type-directed partial evaluation reaches sums only with control and stops at inductive types [Danvy 1996]. We show, inside weak combinatory logic and with mechanized proofs, that the behavioral question has a different answer, one that turns on where the value must be total (Table 1). A function has a *modulus* when it reads its argument only to a bounded depth: boundedly many successor layers of a numeral, or boundedly many bits of a stream. A modulus is an operational form of totality on a compact point set: it asks for a value on every term that agrees with a point up to a finite depth. For $\texttt{Nat} \to \texttt{Bool}$ the points correspond to

Author's Contact Information: N. E. Davis, NDavis@ZeroAttest.com, ZeroAttest, Champaign, Illinois, USA.

2026. ACM 1557-945X/2026/11-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Table 1. Behavioral lifts, by type and by where totality is demanded.

	totality on a non-compact point set	totality on a compact point set
`Bool` → `Bool`	—	lift (Thm. 4.1)
`Nat` → `Bool`	$\mathbb{N}$: no lift (Thm. 5.1)	$\mathbb{N}_\infty$: lift (Thms. 4.4, 5.5)
Cantor space[§]	definable streams: no lift (Thm. 5.6)	all points: lift[‡] (Thms. 4.7, 5.7)

[§]functionals of streams. At (`Nat` → `Bool`) → `Bool` only the lift under a modulus is mechanized (Thm. 4.7).
[‡]in an oracle semantics, by the classical fan theorem.

$\mathbb{N}_\infty$, the naturals with a point at infinity; for functionals of streams they are the points of Cantor space, the infinite boolean sequences $\alpha : \mathbb{N} \to$ `Bool`, of which terms define only countably many. Totality on the numerals does not demand a value at ∞, and under that hypothesis no lift exists (Theorem 5.1).

The failure is sharp: one function that is total on every numeral but has no value at ∞, added to the functions with a modulus, defeats every lift (Theorem 5.4). The obstruction is intensional; extensionally, the functions with a modulus are eventually constant and the witness is constantly false. The *staging line* can be read as the compactness of the point set on which totality is demanded. This recapitulates Escardó's boundary between compact and discrete types for decidable equality [Escardó 2008, Thm. 4.13], but for code extraction.

Contributions.

(1) The S4 reading of compile-time quotation in $\{S, K, I\}$, with re-encoding and intensional fixed points mechanized, and a definition of behavioral lift parameterized by a hypothesis and a set of test inputs (§3).
(2) Lifts from a modulus, inside weak combinatory logic: at `Bool` → `Bool`, at `Nat` → `Bool`, and at Cantor space, each returning a canonical code and each idempotent, so that Kavvos' second coalgebra law holds up to conversion and fails at his ∼ (§4).
(3) When totality yields a modulus (§5): on the numerals it does not, and no lift exists, by a stuttering simulation; on the points of $\mathbb{N}_\infty$ it does outright; on the definable streams it does not, by a Kleene tree built inside the calculus; and on all points of Cantor space it does, in an oracle semantics.
(4) Consequences for a staged language's data rule, including a fix to SKIjack's checker (§6).

Every theorem is mechanized in Lean 4 against `lean-ski` [Davis 2026a] (§7). The ingredients are known: finite tabulation [Altenkirch and Uustalu 2004], Berger's search [Berger 1990] and Escardó's account of it [Escardó 2007a, 2008], and the continuity argument behind finite identification [Balbach 2020]. Section 8 says what is theirs.

2 Preliminaries

Terms and reduction. Terms are S, K, I and applications $t\,u$, associating to the left; all terms are closed. Weak reduction $\twoheadrightarrow$ is the reflexive–transitive closure of $S\,f\,g\,x \to f\,x\,(g\,x)$, $K\,x\,y \to x$, $I\,x \to x$ anywhere in a term. Head reduction $\twoheadrightarrow_h$ contracts only at the head of the spine. A term is in weak head normal form when its head combinator has too few arguments to contract. We use three standard facts, all mechanized: Church–Rosser for $\twoheadrightarrow$ [Hindley and Seldin 2008, Thm. 2.15] (`church_rosser`); a form of standardization [Curry et al. 1972, §11B3], [Hindley and Seldin 2008, Rem. 3.24], by which every weak reduction to a weak head normal form yields a head reduction to one (`eval_back`, `Compose.eval_K_iff`); and determinism of head reduction, which is immediate because a term has at most one head redex.

Booleans, numerals, streams, codes. $\mathsf{T} = K$ and $\mathsf{F} = K\,I$; they are normal and distinct, so by Church–Rosser no term reduces to both. A boolean selects: $\mathsf{T}\,x\,y \twoheadrightarrow x$ and $\mathsf{F}\,x\,y \twoheadrightarrow y$. Scott numerals are $\underline{0} = K$ and $\underline{m+1} = \mathrm{Succ}\,\underline{m}$ with $\mathrm{Succ} = S\,(K\,K)\,(S\,(K\,(S\,I))\,K)$, so that $\mathrm{Succ}\,x\,z\,s \twoheadrightarrow_h s\,(K\,x\,s) \twoheadrightarrow s\,x$. Streams are Scott pairs: $\mathrm{Cons}\,h\,t\,k \twoheadrightarrow k\,h\,t$, with $\mathrm{Hd}\,s = s\,\mathsf{T}$ and $\mathrm{Tl}\,s = s\,\mathsf{F}$. The code $\langle t\rangle$ of a term is its Scott encoding over the object type $S \mid K \mid I \mid \mathrm{App}$ with four continuations (`quoteT`), so $\langle f\,a\rangle = \mathrm{App}\,\langle f\rangle\,\langle a\rangle$ literally. For a fixed code c, a term EqT_c decides equality with c: $\mathrm{EqT}_c\,\langle d\rangle \twoheadrightarrow (c = d)$. We build it by recursion on c in the metalanguage (`eqT_spec`).

Recursion. Recursive terms are fixed points of a step function F, written $\mathrm{FixSelf}\,F\,z$ after `lean-ski`. Each unrolling wraps the self-reference in one more I, so the self-references are not syntactically equal; the invariant is that each still head-reduces to $\mathrm{FixB}\,F$ (`fixSelf_unroll`).

Infinite numerals. Let ω range over the self-references $\mathrm{FixSelf}\,\mathrm{Succ}\,z$ with $z \twoheadrightarrow_h \mathrm{FixB}\,\mathrm{Succ}$: the *infinite numerals*. With two arguments, $\omega\,a\,s \twoheadrightarrow_h s\,(K\,\omega'\,s)$ for another infinite numeral ω', in one explicit step followed by a head reduction (`omega_steps`), exactly as $\underline{m+1}\,a\,s \twoheadrightarrow_h s\,(K\,\underline{m}\,s)$; with fewer arguments ω reaches a weak head normal form other than K (`omega_whnf0`, `omega_whnf1`). We write ω_0 for $\mathrm{FixSelf}\,\mathrm{Succ}$ ($\mathrm{FixB}\,\mathrm{Succ}$); the results below about a value at an infinite numeral are stated, and mechanized, at ω_0.

Prefixes and layers. A term s *has the prefix* $w = b_0 \cdots b_{k-1}$ when $s \twoheadrightarrow \mathrm{Cons}\,b_0\,s_1$, $s_1 \twoheadrightarrow \mathrm{Cons}\,b_1\,s_2$, …, $s_{k-1} \twoheadrightarrow \mathrm{Cons}\,b_{k-1}\,s_k$ for some s_i, the last tail arbitrary (`HasPrefix`). A term n *has at least* m *layers* when $n \twoheadrightarrow \mathrm{Succ}\,n_1$, $n_1 \twoheadrightarrow \mathrm{Succ}\,n_2$, …, m times, the last tail arbitrary (`AtLeast`). Both are closed under backward reduction, and $\underline{c}$ has at least m layers when $c \geq m$.

3 Staging and behavioral lifts

Compile-time quotation makes $\Box$ a comonad without a unit (§3.1). A behavioral lift is a weak substitute for the missing unit (§3.2).

3.1 The staging modality

S4 is the modal logic with the axioms T, $\Box A \to A$ (what is necessary is true), and 4, $\Box A \to \Box\Box A$ (if something is necessary, it is necessarily necessary). Davies and Pfenning read $\Box A$ as codes of terms of type A [Davies and Pfenning 2001]: T is then evaluation of code, and 4 re-encodes a code as the code of that code. The converse of T, $A \to \Box A$, which S4 does not prove, would quote a live value. Over $\{S, K, I\}$ the first two are definable and the third is not (Table 2).

Table 2. The S4 reading of staging, and what $\{S, K, I\}$ defines.

axiom	map	over $\{S, K, I\}$
T: $\Box A \to A$	unquote, ε	$\varepsilon\,\langle t\rangle \twoheadrightarrow t$, definable (`Unquote.unquote`)
4: $\Box A \to \Box\Box A$	re-encode, δ	$\delta\,\langle t\rangle \twoheadrightarrow \langle\langle t\rangle\rangle$, definable (Prop. 3.1)
converse of T: $A \to \Box A$	quote, η	no closed Q has $Q\,M \twoheadrightarrow \langle M\rangle$ [Barendregt 1991]

An unquoter $\varepsilon : \Box A \to A$ with $\varepsilon\,\langle t\rangle \twoheadrightarrow t$ is definable, as the fixed point of

$$\lambda e\,q.\ q\,S\,K\,I\,(\lambda f\,a.\ e\,f\,(e\,a))$$

(`Unquote.unquote`); SKIjack's encodes one in 43 atoms [Davis 2027]. Re-encoding is definable as well since it acts on data:

Proposition 3.1 (`Requote.requote`). *There is a closed R with $R\,\langle t\rangle \twoheadrightarrow \langle\langle t\rangle\rangle$ for every t.*

Proof. R is the fixed point of

$$\lambda r\, q.\ q\,\langle\langle S\rangle\rangle\,\langle\langle K\rangle\rangle\,\langle\langle I\rangle\rangle\,(\lambda f\, a.\ \mathrm{App}'\,(r\,f)\,(r\,a)),$$

where $\mathrm{App}'\,u\,v = \mathrm{App}\,(\mathrm{App}\,\langle\mathrm{App}\rangle\,u)\,v$ is the code of an application given the codes of its parts. Induction on t. □

With ε and $\delta = R$, the counit laws hold as far as codes allow: $\varepsilon \circ \delta = \mathrm{id}$ outright (`Unquote.counit_delta`), and $\Box\varepsilon \circ \delta = \mathrm{id}$ only up to conversion under the box, since $\Box\varepsilon\,(\delta\,\langle t\rangle)$ reduces to the code of $\varepsilon\,\langle t\rangle$, a term that reduces to t (`Unquote.box_counit_delta`). The same gap recurs in the second coalgebra law (§4.4). The unit $\eta : A \to \Box A$, a closed Q with $Q\,M \twoheadrightarrow \langle M\rangle$, is exactly what no term provides [Barendregt 1991] (`no_quote`). Kavvos [Kavvos 2017, §1.3–1.4] describes the same situation for untyped λ. Self-reference survives the missing unit:

Theorem 3.2 (Intensional fixed points; `Ifix.ifix`, `Ifix.quine`). *For every g there is t with $t \twoheadrightarrow g\,\langle t\rangle$. In particular there is a quine, a term t with $t \twoheadrightarrow \langle t\rangle$.*

Proof. Let $h = \lambda y.\ g\,(\mathrm{App}\,y\,(R\,y))$ and $t = h\,\langle h\rangle$. Then

$$t \twoheadrightarrow g\,(\mathrm{App}\,\langle h\rangle\,(R\,\langle h\rangle)) \twoheadrightarrow g\,(\mathrm{App}\,\langle h\rangle\,\langle\langle h\rangle\rangle) = g\,\langle h\,\langle h\rangle\rangle = g\,\langle t\rangle.$$

The only quotation is the compile-time code of the closed term h. □

This is Kavvos' intensional fixed-point rule [Kavvos 2017], realized by re-encoding.

3.2 Behavioral lifts

Definition 3.3 (Lift). Fix a set X of test inputs and a hypothesis H on terms. A closed term L *lifts* under H on X when for every f with $H(f)$ there is a term c with

$$L\,f \twoheadrightarrow \langle c\rangle \qquad \text{and} \qquad \forall x \in X,\ \forall r \in \{\mathsf{T}, \mathsf{F}\},\ \ f\,x \twoheadrightarrow r \implies c\,x \twoheadrightarrow r.$$

The definition is as weak as the informal notion allows: L need work only on terms satisfying H; c may be any term and need be neither f nor canonical; agreement is required only on X. The impossibility (§5.1) refutes even the weaker form in which $L\,f$ and $\langle c\rangle$ need only have a common reduct. The impossibility is not a corollary of the no-quote theorem, which concerns syntax and every term, total or not; the finite lift below exists although that theorem holds.

The instances are listed in Table 1, with the test inputs for $\mathtt{Bool} \to \mathtt{Bool}$ as T, F and for $\mathtt{Nat} \to \mathtt{Bool}$ as the numerals. They are, at Cantor space, the terms with a prefix of length m under a modulus, and the definable streams under totality.

A *Q-coalgebra* in Kavvos' sense [Kavvos 2017, Def. 33] is a map $\alpha : A \to \Box A$ with $\varepsilon \circ \alpha \sim \mathrm{id}$ and $\delta \circ \alpha \sim \Box\alpha \circ \alpha$. The first law, with $\sim$ read as agreement on the test inputs, is satisfied by the lifts of §4; the second holds only up to conversion of the quoted terms, and fails at his $\sim$ (§4.4). Our lifts are therefore not Q-coalgebras in his sense but a weak analogue of them.

4 Lifts from a modulus

Each lift in this section has the same shape: a search decides properties of the function's finite table, a tabulation turns the table into a canonical tree, and the lift returns the code of an evaluator for that tree. The finite case needs no search, since its domain is listed.

4.1 The finite case

Theorem 4.1 (`LiftBool.lift_bool`, `lift_canonical`). *The term*

$$L_{\mathtt{Bool}} = \lambda f.\ f\,\mathsf{T}\,(f\,\mathsf{F}\,\langle K\,K\rangle\,\langle I\rangle)\,(f\,\mathsf{F}\,\langle\mathrm{Not}\rangle\,\langle K\,(K\,I)\rangle)$$

lifts under totality on $\{\mathsf{T}, \mathsf{F}\}$. Functions with the same values get the same code.

Proof. If $f\,\mathsf{T} \twoheadrightarrow t$ and $f\,\mathsf{F} \twoheadrightarrow e$, two selections give the code of the canonical function with values t, e: $K\,K$ (both true), I, Not, or $K\,(K\,I)$ (both false). Each agrees with f on T and F, because a term reaches at most one boolean. □

This is routine; Altenkirch and Uustalu [2004] define the same tabulation at every finite type, in the metalanguage.

4.2 At Nat → Bool

Definition 4.2 (m-local; `NatLift.NLocal`*).* f is *m-local* with values $\varphi(0), \ldots, \varphi(m)$ when $f\,\underline{c} \twoheadrightarrow \varphi(c)$ for $c < m$, and $f\,n \twoheadrightarrow \varphi(m)$ for every term n with at least m layers.

An m-local f forces at most m successor layers of its argument, and f has a *modulus* when it is m-local for some m. An m-local f is total on the numerals (`nlocal_total`) and reaches $\varphi(m)$ on an infinite numeral (`omega_atLeast`), so it is total on the points of $\mathbb{N}_\infty$, with value $\varphi(m)$ at ∞.

Search. Let $\mathrm{Srch}\,p = p\,\underline{0}\,\underline{0}\,(\mathrm{Succ}\,(\mathrm{Srch}\,(p \circ \mathrm{Succ})))$: if $p\,\underline{0}$ then $\underline{0}$, else the successor of the search for $p \circ \mathrm{Succ}$. It builds a lazy numeral, infinite when p is never true.

Lemma 4.3 (Search over $\mathbb{N}_\infty$; `search_nat`). *If p is m-local with values φ, then $p\,(\mathrm{Srch}\,p)$ reduces to* T *iff $\varphi(k)$ holds for some $k \le m$, and to* F *otherwise.*

Proof. Induction on m. For $m = 0$, p is constant. For $m+1$, $p \circ \mathrm{Succ}$ is m-local with values $\varphi(\cdot + 1)$ (`nlocal_shift`). If $\varphi(0)$, $\mathrm{Srch}\,p \twoheadrightarrow \underline{0}$ and $p\,\underline{0} \twoheadrightarrow \mathsf{T}$. Otherwise $\mathrm{Srch}\,p \twoheadrightarrow \mathrm{Succ}\,(\mathrm{Srch}\,(p \circ \mathrm{Succ}))$, and $p\,(\mathrm{Succ}\,x)$ and $(p \circ \mathrm{Succ})\,x$ have a common reduct, so the induction hypothesis and Church–Rosser give the result. □

This search corresponds to Escardó's for $\mathbb{N}_\infty$ [Escardó 2008] but run on Scott numerals. m-locality lets p terminate on the lazy numeral built by the search.

Tabulation. The search yields two deciders. The first, $\mathrm{Every}\,p = \mathrm{Not}\,(p'\,(\mathrm{Srch}\,p'))$ with $p' = \mathrm{Not} \circ p$, decides whether φ holds at every $k \le m$ (`everyN_spec`). The second,

$$\mathrm{Const}\,f = \mathrm{Every}\,(\lambda n.\ \mathrm{EqB}\,(f\,n)\,(f\,\omega_0)),$$

decides whether f is constant (`constN_spec`). Then

$$\mathrm{Tree}\,f = \mathrm{Const}\,f\ (\mathrm{Leaf}\,(f\,\omega_0))\ (\mathrm{Node}\,(f\,\underline{0})\,(\mathrm{Tree}\,(f \circ \mathrm{Succ})))$$

reduces to a datum $D_m(\varphi)$ defined by recursion on m: a leaf of $\varphi(m)$ if φ is constant on $0..m$, else a node of $\varphi(0)$ and $D_{m-1}(\varphi(\cdot + 1))$ (`treeN_spec`). It depends only on φ (`DN_congr`). The evaluator

$$\mathrm{Eval}\,d\,n = d\,(\lambda b.\,b)\,(\lambda b\,t.\ n\,b\,(\lambda n'.\ \mathrm{Eval}\,t\,n'))$$

runs the datum: $\mathrm{Eval}\,D_m(\varphi)\,\underline{c} \twoheadrightarrow \varphi(c)$ for $c < m$ and $\mathrm{Eval}\,D_m(\varphi)\,n \twoheadrightarrow \varphi(m)$ for n with at least m layers (`evalN_spec`). The structural lift LiftTree at the tree's data type sends the datum to its own code (`liftTreeN_spec`).

Theorem 4.4 (`NatLift.nat_lift`, `nat_canonical`). *The term*

$$L_{\mathsf{Nat}} = \lambda f.\ \mathrm{App}\,\langle \mathrm{Eval}\rangle\,(\mathrm{LiftTree}\,(\mathrm{Tree}\,f))$$

lifts every m-local f, for every m, on the numerals: $L_{\mathsf{Nat}}\,f \twoheadrightarrow \langle \mathrm{Eval}\,D_m(\varphi)\rangle$, and that term agrees with f on every numeral. Two m-local functions with the same values get the same code.

Proof. Compose the lemmas; agreement on $\underline{c}$ with $c \ge m$ uses that $\underline{c}$ has at least m layers, and that a term reaches at most one boolean. □

4.3 At Cantor space

Definition 4.5 (m-local, streams; `Search.Local`*).* q is *m-local* with table $\varphi : \{\mathsf{T},\mathsf{F}\}^m \to \{\mathsf{T},\mathsf{F}\}$ when $q\,s \twoheadrightarrow \varphi(w)$ for every term s with a prefix w of length m.

As at $\mathtt{Nat} \to \mathtt{Bool}$, q has a *modulus* when it is m-local for some m.

Berger's search. The search [Berger 1990] is

$$\mathrm{Find}\,p = (\lambda l.\ p\,l\,l\,(\mathrm{Cons}\,\mathsf{T}\,(\mathrm{Find}\,(p \circ \mathrm{Cons}\,\mathsf{T}))))\ (\mathrm{Cons}\,\mathsf{F}\,(\mathrm{Find}\,(p \circ \mathrm{Cons}\,\mathsf{F}))).$$

Lemma 4.6 (`Search.search`, `search_decides`). *If p is k-local with table φ, then* $\mathrm{Find}\,p$ *has a prefix u of length k, $p\,(\mathrm{Find}\,p) \twoheadrightarrow \varphi(u)$, and $\varphi(u)$ holds whenever φ holds at some word of length k.*

Proof. Induction on k. For $k = 0$, p is constant, so $\mathrm{Find}\,p$ reduces to one of the two cons cells. For $k+1$, $p \circ \mathrm{Cons}\,b$ is k-local with table $\varphi(b\,\cdot)$ (`local_pre`). By induction $\mathrm{Find}\,(p \circ \mathrm{Cons}\,\mathsf{F})$ has a prefix u_0, so $l = \mathrm{Cons}\,\mathsf{F}\,(\cdots)$ has the prefix $\mathsf{F}u_0$, and $p\,l \twoheadrightarrow \varphi(\mathsf{F}u_0)$. If that is T, $\mathrm{Find}\,p \twoheadrightarrow l$. Otherwise $\varphi(\mathsf{F}\,\cdot)$ is constantly F (by the induction hypothesis for the left branch), and $\mathrm{Find}\,p$ reduces to the right cons cell, whose prefix $\mathsf{T}u_1$ satisfies φ if anything starting with T does. □

We claim no termination of Find but do not require it; the lemma asserts only that reductions exist. With sharing, $\mathrm{Find}\,p$ calls p at most $2^{k+1} - 1$ times, and a predicate that reads its bits last-first comes close to that bound; this is an informal remark about an implementation with sharing, outside the reduction semantics of §2, and is not mechanized.

Tabulation and lift. As in §4.2: Every and Const (against the zero stream $0^\omega = \mathrm{Tab}\,(K\,\mathsf{F})$) decide universality and constancy (`every_spec`, `const_spec`);

$$\mathrm{Tree}\,q = \mathrm{Const}\,q\,(\mathrm{Leaf}\,(q\,0^\omega))\,(\mathrm{Node}\,(\mathrm{Tree}\,(q \circ \mathrm{Cons}\,\mathsf{F}))\,(\mathrm{Tree}\,(q \circ \mathrm{Cons}\,\mathsf{T})))$$

reduces to a datum $D_m(\varphi)$ depending only on φ (`tree_spec`, `D_congr`); an evaluator reads the stream's bits down the tree (`eval_spec`); and LiftTree sends the tree to its code (`liftTree_spec`).

Theorem 4.7 (`CantorLift.cantor_lift`, `cantor_canonical`, `fun_lift`). *The term*

$$L_C = \lambda q.\ \mathrm{App}\,\langle\mathrm{Eval}\rangle\,(\mathrm{LiftTree}\,(\mathrm{Tree}\,q))$$

lifts every m-local stream functional, for every m, on the terms with a prefix of length m. Two m-local functionals with the same table get the same code. A variant, which applies L_C to $\Phi \circ \mathrm{Nth}$ and precomposes the resulting code with Tab*, lifts every functional Φ of functions on numerals that reads its argument at $0, \ldots, m-1$ only, agreeing with it on every f whose values there are booleans (*`flocal_stream`*, *`tab_spec`*).*

While the search and the tabulation are due to Berger and Escardó [Berger 1990; Escardó 2007b, 2008], the short step from them to a finite canonical code is, as far as we know, novel.

4.4 The second coalgebra law

Our lifts satisfy Kavvos' second law up to conversion of the quoted terms, but not with his $\sim$ at $\Box\Box A$. For a lift α with $\alpha\,f \twoheadrightarrow \langle c\rangle$, $\delta(\alpha\,f) \twoheadrightarrow \langle\langle c\rangle\rangle$ and $(\Box\alpha)(\alpha\,f) = \mathrm{App}\,\langle\alpha\rangle\,(\alpha\,f) \twoheadrightarrow \langle\alpha\,c\rangle$. Each of $L_{\mathtt{Bool}}$, $L_{\mathtt{Nat}}$ and L_C is idempotent on its outputs, $\alpha\,c \twoheadrightarrow \langle c\rangle$: the output term meets the lift's own hypothesis with the same table, and canonicality returns the same code (`Coalgebra.bool_idem`, `nat_idem`, `cantor_idem`). By idempotence the two sides of the law are codes of convertible terms (`Coalgebra.coalg2`). Kavvos' $\sim$ at $\Box\Box A$ is stricter. An object is *point-univalent* when its extensionally equal points are intensionally equal [Kavvos 2017, Def. 26], and every object of codes is [Kavvos 2017, Lemma 18], so at $\Box\Box A$ his $\sim$ compares codes as codes. The law fails there: $\langle\langle c\rangle\rangle$ and $\langle\alpha\,c\rangle$ are codes of distinct terms, since a code is never an application to the term it encodes,

and the equality test EqT of §2 separates them (`Coalgebra.coalg2_strict_fails`). His results on Q-coalgebras, which assume the law at that $\sim$ [Kavvos 2017, Lemmas 19–20, Thms. 30–31], therefore do not apply to our lifts.

5 Totality and compactness

Section 4 lifts every function with a modulus. This section asks when totality gives a modulus. On the numerals it does not, and no lift exists (§5.1, §5.2). On the points of $\mathbb{N}_\infty$ it does outright (§5.3). At Cantor space it does not on the definable streams, and it does on all points in an oracle semantics (§5.4). One simulation argument, Lemma 5.3 and its generalizations, drives every cell.

5.1 No lift under totality on the numerals

Theorem 5.1 (`NoLift.no_lift_conv`). *No closed L sends every f total on the numerals to a term with a common reduct with a code $\langle c\rangle$ such that c agrees with f on every numeral.*

Ingredients. Let $G\,c\,n = n\,\mathsf{F}\,(\lambda n'.\ c\,\mathsf{T}\,(\lambda c'.\ G\,c'\,n'))$, which decides $n > c$ by walking down both numerals: $G\,\underline{j}\,\underline{n} \twoheadrightarrow (j < n)$ (`G_num`) and $G\,\omega\,\underline{n} \twoheadrightarrow \mathsf{F}$ for all n (`G_omega`). The equality test EqT_c of §2 suffices because the proof fixes c first.

Definition 5.2. $\mathrm{Rel}_b\ t\ t'$ holds when t' is t with some infinite numerals replaced by numerals $\underline{m}$, $m \geq b$: it is the congruence generated by identity and the pairs $(\omega, \underline{m})$, $m \geq b$.

Lemma 5.3 (Stuttering simulation; `NoLift.sim`). *If $t \twoheadrightarrow_h K$ in n steps, $n < b$, and $\mathrm{Rel}_b\ t\ t'$, then $t' \twoheadrightarrow_h K$.*

The simulation is *stuttering*: it matches runs, not single steps, as in the simulation diagrams of compiler verification [Leroy 2009]. At a combinator head both sides contract the same redex. At an infinite-numeral head, t unfolds ω and t' unfolds $\underline{m}$, by head reductions whose lengths may differ, and the bound b drops by one. The bound counts how many more unfoldings of ω the numerals $\underline{m}$, $m \geq b$, can match.

Proof. Induction on n. A related pair has a common shape: the same combinator at the head of both spines with pointwise related arguments, or ω against $\underline{m}$ with related arguments (`Rel.spine`). *Combinator heads.* If the head is S, K or I with enough arguments, both sides contract the same redex and the results are related at the same b (a congruence tolerates duplication by S and erasure by K). With too few arguments t is a weak head normal form, so $n = 0$ and $t = K$, whence $t' = K$. *Infinite-numeral heads.* With two or more arguments, t takes at least one step to $s\,(K\,\omega'\,s)\,\vec{r}$ and, since $m \geq b > n \geq 1$, $\underline{m} = \underline{k+1}$ reduces to $s'\,(K\,\underline{k}\,s')\,\vec{r}\,'$. The results are related at $b-1$, and the remaining run from t's reduct is shorter than $b-1$ steps. With fewer arguments t's run would stop at a weak head normal form other than K, which determinism of head reduction rules out. □

Proof of Theorem 5.1. Suppose L lifts. The function $f = G\,\omega_0$ is total (constantly F); let $\langle c\rangle$ be its code. Then $\mathrm{EqT}_c\,(L\,f) \twoheadrightarrow \mathsf{T}$, so $\mathrm{EqT}_c\,(L\,f)$ head-reduces to K in some n steps. The term $\mathrm{EqT}_c\,(L\,(G\,\underline{n+1}))$ is related to it at $n+1$, so it too reduces to T (Lemma 5.3): L gives the total function $G\,\underline{n+1}$ the same code c. But $G\,\omega_0\,\underline{n+2} \twoheadrightarrow \mathsf{F}$ while $G\,\underline{n+1}\,\underline{n+2} \twoheadrightarrow \mathsf{T}$, so $c\,\underline{n+2}$ reaches both booleans, contradicting Church–Rosser. □

The argument is the classical one that the almost-everywhere-zero functions are not finitely identifiable [Balbach 2020]: an answer committed to after finitely many observations of the zero function is shared by a function that differs later. The simulation supplies "finitely many observations" for a live argument.

5.2 Sharpness

The proof of Theorem 5.1 uses only $G\,\omega_0$ and the $G\,\underline{k}$ (`no_lift_core`), and each $G\,\underline{k}$ has a modulus: it is $(k{+}1)$-local (`G_num_nlocal`). So Theorems 5.1 and 4.4 combine:

Theorem 5.4 (`G_omega_not_local`, `no_lift_local_plus_one`). *$G\,\omega_0$ is m-local for no m, and no closed term lifts every m-local function (for every m) together with $G\,\omega_0$.*

Proof. If $G\,\omega_0$ were m-local, L_{Nat} would lift it and every $G\,\underline{k}$, contradicting `no_lift_core`; the second claim is the same argument for any L. □

So among the functions total on the numerals, the line between lift and no lift is totality at ∞: adding $G\,\omega_0$, one function without a value there, defeats every lift.

5.3 At $\mathbb{N}_\infty$

Totality on the points of $\mathbb{N}_\infty$ gives a modulus outright. The points of $\mathbb{N}_\infty$ are definable: the numerals and an infinite numeral ω. Lemma 5.3 generalizes in two ways (`NatFan.simW`): an infinite numeral may be related to *any* term with enough successor layers, the right side reducing weakly; and the run may end at any weak head normal form v, returning a related v'. The second form serves the false case: a run of $f\,\omega_0$ to F ends at some $K\,x$ with $x \twoheadrightarrow I$, and the simulation then follows x's run in turn.

Theorem 5.5 (`NatFan.nat_fan`, `local_iff_total_inf`, `nat_lift_inf'`). *A function total on the numerals and at ω_0 is m-local for some m. Hence, for f total on the numerals, f has a modulus iff $f\,\omega_0$ reaches a boolean, and L_{Nat} lifts every f total on the points of $\mathbb{N}_\infty$.*

Proof. If $f\,\omega_0$ head-reduces to a boolean in n steps (plus n' for the inner I in the false case), the simulation fixes f's value on every term with at least $n + n' + 2$ layers. The values on the finitely many numerals below that depth come from totality by finite choice, which needs no axiom of choice. □

With Theorem 5.4, the `Nat` row is settled as a statement about classes: L_{Nat} lifts every function total on the numerals and at ∞, and no closed term lifts that class together with $G\,\omega_0$, which is total on the numerals and has no value at ω_0 (`NatFan.G_omega_no_value`).

5.4 At Cantor space

At Cantor space the answer depends on which points count: totality on the definable streams gives no lift, and totality on all points gives one in an oracle semantics.

Under totality on the definable streams. The impossibility of §5.1 holds for any family $\Psi\,\omega$, $\Psi\,\underline{k}$ that some test input separates for every k (`CantorTotal.no_lift_family`). Take a binary tree T of finite words, decided on stream prefixes by a closed term, and let $\Psi\,c\,s$ ask whether s leaves T within c steps. $\Psi\,\underline{k}$ gives up after k steps; $\Psi\,\omega$ never gives up. If T has no definable infinite path, $\Psi\,\omega$ is total and true on every definable stream. If T is infinite and prefix-closed, a stream that follows a long word of T and then continues with zeros separates $\Psi\,\omega$ from $\Psi\,\underline{k}$.

Theorem 5.6 (`KleeneTree.no_lift_cantor_total'`). *No closed term lifts every functional total on the definable streams.*

The proof needs a Kleene tree decided by a closed term, and we build one inside the calculus (`KleeneTree.kleene`). A small universal machine on codes performs one head step (`Univ.stepC_spec`, mirroring the computable contraction) and a bounded run (`evalC_spec`); a diagonal walk numbers every term (`dec_surj`, `decC_num`). The tree holds a word w iff, for every $e < |w|$, the

term numbered e, applied to $\underline{e}$ and read as a boolean, does not reach $w(e)$ within $|w|$ steps. It is prefix-closed (a verdict found with less fuel stands with more), infinite (choose each bit against its verdict), and has no definable path: a definable stream s gives the term Nth s, whose number e has a run converging to s's bit at e.

At Cantor space, with oracles. The step from pointwise totality to a modulus, the *fan step*, needs non-definable points. Extend the calculus with a constant Ora_α for every $\alpha : \mathbb{N} \to \mathtt{Bool}$, reducing as a stream cell: $\mathrm{Ora}_\alpha\, k \;\to\; k\,\alpha(0)\,\mathrm{Ora}_{\alpha(\cdot+1)}$. We call the extended calculus the *oracle semantics*: every point of Cantor space, definable or not, is now a term. A closed functional is *oracle-total* when $F\,\mathrm{Ora}_\alpha$ head-reduces to exactly T or F for every α. For F this asks more than the proof needs; the relaxed form (`OracleFan.cantor_fan_w`) uses the two-stage argument of the $\mathbb{N}_\infty$ case. A term may apply a stream to arbitrary continuations, not only to the head and tail selectors, but the simulation absorbs this, because the oracle answers any continuation as a cons cell would. So a run of $F\,\mathrm{Ora}_\alpha$ is mirrored on every term of the pure calculus that shares enough of α's bits, whatever its tail (`OracleFan.sim0`). Hence F has a *pointwise modulus* at every α: a length n_α such that F returns its value at α on every term that shares the first n_α bits of α (`oracle_local`).

Theorem 5.7 (`OracleFan.cantor_fan`, `cantor_lift_oracle`). *An oracle-total functional is m-local for some m, and L_C lifts it, agreeing with it on every term with a prefix of length m, in particular on every definable stream.*

Proof. The fan theorem (`OracleFan.fan`, proved by König's lemma) turns the pointwise moduli into one bound. The fan theorem is classical: these results use `Classical.choice`, and the simulation does not. □

Where the step is classical. Call F *barred* at a word w when it is constant on every term with prefix w, or when it is barred at both one-bit extensions of w. This is an inductive bar, the statement that F's query tree is well-founded. Constructively, being barred at the empty word is equivalent to having a modulus (`OracleBar.obar_iff_local`). The fan step then factors into three parts (`cantor_fan_bar`):

(1) from runs to pointwise moduli, by the simulation (constructive);
(2) from pointwise moduli to a bar, by the fan theorem for this bar (`pointwise_obar`; the only classical step of the factorization);
(3) from the bar to a table (constructive).

Passing to the term model does not remove the middle step; it makes the step fail. Theorem 5.6 records the failure: over the definable streams, pointwise totality gives no bar (a bar would give a modulus, `obar_iff_local`, and so a lift, against Theorem 5.6), because the Kleene tree above is infinite and has no definable path.

5.5 Scope

Sections 4 and 5 settle Table 1, using classical logic only for the fan theorem in the oracle cell (`OracleFan.fan`, proved with `Classical.choice`). An open question remains in the `Nat` row: if adding *any* function without a value at ∞ to the functions with a modulus defeats every lift.

6 Consequences for staged languages

A staged language's data rule should refuse provable functions and admit lifted codes, which are data. A *provable function* is a term that the checker can show, from its syntax and the declared types, to be a function: a lambda, a namespace literal, or the name of a macro or a core; a combinator, constructor or equation applied to fewer arguments than it takes; or an equation applied to exactly

as many, when its body is a provable function. The guarantee is one-directional: it does not assert that every accepted operand is a datum [Davis 2026b]. SKIjack enforces the rule in two stages [Davis 2026b], the first of which is fixed here. Stage A (which formerly decided "function" by the presence of binders) decides by arity, and refuses an under-applied combinator or equation, or a saturated equation whose body is one. Stage B, a Hindley–Milner discipline over the declared sum types, refuses a function that reaches an operand of `EQ` through a bound variable. With both stages, no accepted program has a provable function as an operand of `EQ`; namespace facts and scry paths are data by their syntax. These are properties of the SKIjack implementation, tested there, not theorems mechanized here.

Because lifted codes are data, `EQ` on two of them legitimately decides extensional equality of two live functions at the liftable types. The staging line is not a confidentiality boundary: a term that holds a function can already probe it, and a canonical code reveals only its extension. The same line also appears one level up, in an interpreter hosted on a combinator calculus. SKIlark [Davis 2026c] implements Joy as SKIjack programs: its words are closed supercombinators, and its quotations are Scott lists of words, so a Joy program is data by construction and a word taken from a quotation is a symbol, not a live function. The no-quote theorem binds the host, not the object language: an interpreter quotes its own programs because it represents them. The staging line reappears only where a live host value must become an object-level datum, which is a lift in the sense of §3.2.

The arguments of §§4–5 use confluent weak reduction, deterministic head reduction, and Scott encodings of booleans, numerals, streams and codes. Their case analyses are over S, K and I, but analogous ones should go through for any combinatory basis with these properties, each with its own mechanization. Calculi with an intensional operator differ: SF-calculus and tree calculus [Jay 2021; Jay and Given-Wilson 2011] factorize normal forms, so the premise of the no-quote argument changes there, and where the staging line may fall in them we leave open.

7 Mechanization

All theorems are in `lean-ski` [Davis 2026a], on Lean 4.31; the claims about SKIjack and SKIlark in §6 describe those artifacts and are not theorems. The axiom gate `#audit_clean` admits only `propext` and `Quot.sound`. The theorems we cite are in the gate except the fan theorem and everything resting on it (`OracleFan.fan`, `cantor_fan`, `cantor_fan_w`, `cantor_lift_oracle`, `OracleBar.pointwise_obar`, `cantor_fan_bar`), which use `Classical.choice`. Table 3 places the results by module. We build recursive terms with `lean-ski`'s FixSelf and bracket abstraction over

module	results
`Requote`, `Ifix`	re-encoding; intensional fixed points; a quine
`LiftBool`	Theorem 4.1
`NatLift`	Lemma 4.3, Theorems 4.4 and 5.4
`Search`	Lemma 4.6; Nth, Tab
`CantorLift`	Theorem 4.7
`Coalgebra`	the second coalgebra law
`NoLift`	Theorem 5.1: simulation, G, EqT, `no_lift_core`
`NatFan`	Theorem 5.5
`CantorTotal`, `Univ`, `KleeneTree`	Theorem 5.6
`OracleFan`	Theorem 5.7 (`Classical.choice`)
`OracleBar`	the classical step isolated

Table 3. Where the results are mechanized, by `lean-ski` module.

open terms (`absN`); all reasoning is by existential weak reduction, with head reduction only inside the simulation. The Lean development was written with AI assistance; see the Acknowledgements.

8 Related work

Modality and intensionality. S4 for staging is due to Davies–Pfenning [Davies and Pfenning 2001]. Kavvos [Kavvos 2017, 2020] gave the intensional comonad, Barendregt's no-quote argument, and intensional recursion with the Gödel–Löb axiom [Kavvos 2021]. His Q-coalgebras [Kavvos 2017, Def. 33] are the closest abstract notion; our lifts are a weak analogue of them (§4.4), and he did not characterize which types carry one. Another factorization impossibility for *SK* is due to Jay and Given-Wilson [2011]; Jay's tree calculus quotes programs but not computations [Jay 2021].

Reification. Normalization by evaluation [Berger and Schwichtenberg 1991] and type-directed partial evaluation [Danvy 1996] reify at higher types by type-directed η-expansion over a model with syntax at base type; sums need control or a sheaf model [Altenkirch et al. 2001; Danvy 1996], and inductive types are out of reach. Altenkirch and Uustalu [2004] quote at every finite type of $\lambda^{\to 2}$ by enumerating finite domains as decision trees, in the metalanguage. Our lifts are terms of the calculus applied to live arguments.

Exhaustibility. Berger introduced the search over Cantor space [Berger 1990]. The fan functional computed from a functional alone, decidable equality at compact domains, and the searchability of $\mathbb{N}_\infty$ are due to Escardó [Escardó 2007a,b, 2008]; our reading of the staging line borrows the shape of his compact and discrete types [Escardó 2008, Thm. 4.13]. Decision trees for continuous functionals are also built from a term's syntax, by translation (dialogue trees; see Baillon et al. [2026]); ours come from the live value, by search.

Continuity. It is standard that a computation queries a type-1 argument finitely often, and so cannot extract an index from an oracle [Longley 2005, §2.3–2.4]; see Longley and Normann [2015]. Our specific argument is finite identification [Balbach 2020]. We know of no continuity theorem for weak combinatory logic; combinatory Böhm trees [Dybjer and Kuperberg 2012] would be its natural home, and in λ the continuity and genericity lemmas [Barendregt 1984, §14.3] would give our step. The stuttering simulation is a self-contained substitute.

9 Conclusion

In every cell of Table 1, a behavioral lift exists exactly when totality is demanded on a compact point set. Over $\{S, K, I\}$ the staging line is therefore not drawn by types: `Nat` $\to$ `Bool` lies on both sides of it, according to whether a value at ∞ is required. Each cell is proved and mechanized, under one hypothesis, the classical fan theorem in the oracle semantics, and over the definable streams the Kleene tree shows that the fan step fails. Two questions remain open: whether adding any single function without a value at ∞ to the functions with a modulus defeats every lift, and where the line falls in calculi with an intensional operator, such as SF-calculus and tree calculus.

Acknowledgements

Claude (Anthropic; model Claude Opus 5.5, used through Claude Code) assisted with this work in November 2026: drafting and revising the theorems, writing and checking Lean proofs in `lean-ski`, searching and checking the literature, and checking the claims against the mechanization. The author directed the research, checked all output, and takes responsibility for the content.

Competing interests

The author declares none.

References

Thorsten Altenkirch, Peter Dybjer, Martin Hofmann, and Philip Scott. 2001. Normalization by Evaluation for Typed Lambda Calculus with Coproducts. In *Proceedings of the 16th Annual IEEE Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society, Los Alamitos, CA, 303–310. doi:10.1109/LICS.2001.932506

Thorsten Altenkirch and Tarmo Uustalu. 2004. Normalization by Evaluation for $\lambda^{\rightarrow 2}$. In *Functional and Logic Programming (FLOPS) (LNCS, Vol. 2998)*. Springer, Berlin, 260–275. doi:10.1007/978-3-540-24754-8_19

Martin Baillon, Assia Mahboubi, and Pierre-Marie Pédrot. 2026. In Cantor Space No One Can Hear You Stream. In *Programming Languages and Systems (ESOP 2026), Part I (Lecture Notes in Computer Science)*, Robbert Krebbers (Ed.). Springer, Cham, 72–103. doi:10.1007/978-3-032-22720-1_4

Frank J. Balbach. 2020. Some Classical Results in Inductive Inference of Recursive Functions. *Archive of Formal Proofs* (Aug. 2020). https://isa-afp.org/entries/Inductive_Inference.html, Formal proof development.

Henk Barendregt. 1991. Self-Interpretation in Lambda Calculus. *Journal of Functional Programming* 1, 2 (1991), 229–233.

H. P. Barendregt. 1984. *The Lambda Calculus: Its Syntax and Semantics* (revised ed.). Studies in Logic and the Foundations of Mathematics, Vol. 103. North-Holland, Amsterdam.

Ulrich Berger. 1990. *Totale Objekte und Mengen in der Bereichstheorie*. Ph. D. Dissertation. Ludwig-Maximilians-Universität München. Cited through Escardó (2008).

Ulrich Berger and Helmut Schwichtenberg. 1991. An Inverse of the Evaluation Functional for Typed λ-calculus. In *Proceedings of the Sixth Annual IEEE Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society, Los Alamitos, CA, 203–211. doi:10.1109/LICS.1991.151645

Haskell B. Curry, J. Roger Hindley, and Jonathan P. Seldin. 1972. *Combinatory Logic, Volume II*. North-Holland, Amsterdam.

Olivier Danvy. 1996. Type-Directed Partial Evaluation. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, New York, 242–257.

Rowan Davies and Frank Pfenning. 2001. A Modal Analysis of Staged Computation. *J. ACM* 48, 3 (May 2001), 555–604. doi:10.1145/382780.382785

N. E. Davis. 2026a. lean-ski: SKI combinatory logic in Lean 4. Zenodo, https://doi.org/10.5281/zenodo.24024993. Version 1.0.

N. E. Davis. 2026b. SKIjack. https://github.com/sigilante/SKIjack. Version 0.2.6.

N. E. Davis. 2026c. SKIlark: Joy on SKIjack. https://github.com/sigilante/SKIlark. Version 0.11.0; on PyPI as `skilark-joy`.

N. E. Davis. 2027. A self-interpreter for *SKI*: Authoring semantics for new symbols. (2027). Journal of Functional Programming, in review; doi:10.5281/zenodo.22867957.

Peter Dybjer and Denis Kuperberg. 2012. Formal Neighbourhoods, Combinatory Böhm Trees, and Untyped Normalization by Evaluation. *Annals of Pure and Applied Logic* 163, 2 (2012), 122–131. doi:10.1016/j.apal.2011.06.021

Martín H. Escardó. 2007a. Infinite Sets that Admit Fast Exhaustive Search. In *Proceedings of the 22nd Annual IEEE Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society, Los Alamitos, CA, 443–452. doi:10.1109/LICS.2007.25

Martín H. Escardó. 2007b. Seemingly Impossible Functional Programs. Guest post, *Mathematics and Computation* (A. Bauer's blog).

Martín H. Escardó. 2008. Exhaustible Sets in Higher-Type Computation. *Logical Methods in Computer Science* 4, 3:3 (2008). doi:10.2168/LMCS-4(3:3)2008

J. Roger Hindley and Jonathan P. Seldin. 2008. *Lambda-Calculus and Combinators: An Introduction*. Cambridge University Press, Cambridge.

Barry Jay. 2021. Reflective Programs in Tree Calculus. Self-published.

Barry Jay and Thomas Given-Wilson. 2011. A Combinatory Account of Internal Structure. *Journal of Symbolic Logic* 76, 3 (2011), 807–826.

G. A. Kavvos. 2017. *On the Semantics of Intensionality and Intensional Recursion*. Ph. D. Dissertation. University of Oxford. arXiv:1712.09302.

G. A. Kavvos. 2020. Dual-Context Calculi for Modal Logic. *Logical Methods in Computer Science* 16, 3 (2020).

G. A. Kavvos. 2021. Intensionality, Intensional Recursion, and the Gödel–Löb Axiom. *IfCoLog Journal of Logics and their Applications* 8, 8 (Sept. 2021), 2287–2312. Presented at IMLA 2017; arXiv:1703.01288.

Xavier Leroy. 2009. A Formally Verified Compiler Back-end. *Journal of Automated Reasoning* 43, 4 (2009), 363–446. doi:10.1007/s10817-009-9155-4

John Longley. 2005. Notions of Computability at Higher Types I. In *Logic Colloquium 2000*, René Cori, Alexander Razborov, Stevo Todorčević, and Carol Wood (Eds.). Lecture Notes in Logic, Vol. 19. Association for Symbolic Logic, 32–142.

John Longley and Dag Normann. 2015. *Higher-Order Computability*. Springer, Berlin. doi:10.1007/978-3-662-47992-6

Walid Taha and Tim Sheard. 2000. MetaML and Multi-Stage Programming with Explicit Annotations. *Theoretical Computer Science* 248 (2000), 211–242.